

Mango Bridge

Audit Report

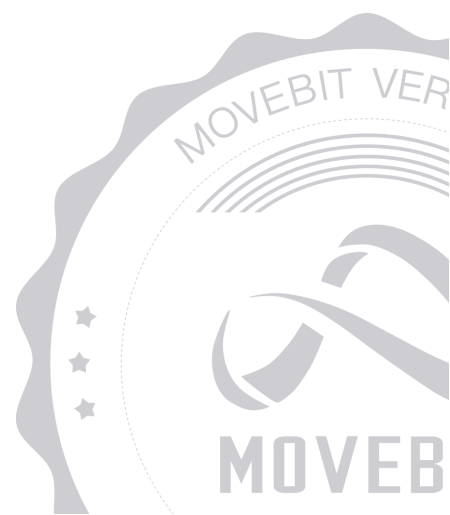


contact@bitslab.xyz



https://twitter.com/movebit_

Thu Feb 06 2025



Mango Bridge Audit Report

1 Executive Summary

1.1 Project Information

Description	A bridge that can bridge tokens between Sui,EVM chain and Mango chain.
Type	Bridge
Auditors	MoveBit
Timeline	Mon Dec 09 2024 - Mon Jan 06 2025
Languages	Move, Solidity
Platform	Others
Methods	Architecture Review, Unit Testing, Manual Review
Source Code	https://github.com/movebit/ClientProjects
Commits	e5d999cca86d5a50d871984bffa152f8e2a35a42 fe248da19d671f035d66609458d4c2a03570e886

1.2 Files in Scope

The following are the SHA1 hashes of the original reviewed files.

ID	File	SHA-1 Hash
MOV	mango-bridge-contracts/sui/Move.toml	576f93617ef5ab046967ead01bb7038bbcb2ced7
BRI	mango-bridge-contracts/sui/sources/bridge.move	fc16944f7c720824ba664640f343c3aa1796cb0c
SIG	mango-bridge-contracts/sui/sources/sign.move	76364f01ab87f7e528ad34c2764fd7b723ddd8e8
CON	mango-bridge-contracts/sui/sources/config.move	e2e93807014d44775afb7dc302b3e9fdf8ca12fd
TRE	mango-bridge-contracts/sui/sources/treasury.move	d13a22cd6e44f19768cee6e3c05b05526071b28e
DEP	mango-bridge-contracts/sui/sources/deposit.move	5e885b98765c6dda3a45c59df5175aae0a21867b
MOV1	mango-bridge-contracts/mango/Move.toml	b58cf729e1fd05c06fc4739baa1a8308aebdda54
BRI1	mango-bridge-contracts/mango/sources/bridge.move	18402a177b3beae3cabdc2f93c33ec27b76a9adc
SIG1	mango-bridge-contracts/mango/sources/sign.move	386e33ad156248a1bd286faf7273ec9048ee989
CON1	mango-bridge-contracts/mango/sources/config.move	5aa596439a77ae0c2d94d786f15a3014d40d41c0
TRE1	mango-bridge-contracts/mango/sources/treasury.move	d245e55b23946b682dbc3dc413defc13b07496a3

DEP1	mango-bridge-contracts/mango/sources/deposit.move	427ded4f555085bb20291f5f751c886dbcd85741
INI	mango-bridge-contracts/solidity/proxy/Utils/Initializable.sol	e5f90b9680ef6a568b63b21f15cf24cb3712ee5d
UUPSU	mango-bridge-contracts/solidity/proxy/Utils/UUPSUpgradeable.sol	575356462b9b43ea60030ce6d6c4037c9bcb2feb
ERC1UU	mango-bridge-contracts/solidity/proxy/ERC1967/ERC1967UpgradeUpgradeable.sol	b45cff2b19f8898b80b8ad56a8d822673f26568
IBU	mango-bridge-contracts/solidity/proxy/beacon/IBeaconUpgradeable.sol	c445b47a51bae223cbf7b47829e4d93f77a3e815
ESU	mango-bridge-contracts/solidity/Utils/EnumerableSetUpgradeable.sol	bd1e9e6518a73e25861fd4fff57c79ef7bb58988
SSU	mango-bridge-contracts/solidity/Utils/StorageSlotUpgradeable.sol	b5f62e31228ce58159c1d63ab7ce5658102d270c
THE	mango-bridge-contracts/solidity/Utils/TransferHelper.sol	ca23dc059c29301e2a6dfbdeb623f30f1ff1c6c0
AUP	mango-bridge-contracts/solidity/Utils/AddressUpgradeable.sol	e6e4c9ec8f1e152a7adc75f3f42ab8576a5ee38b
CUP	mango-bridge-contracts/solidity/Utils/ContextUpgradeable.sol	b0d4033646ff5641469a06d504ef1ce1377c7a54
STR	mango-bridge-contracts/solidity/Utils/Strings.sol	64a06a9e23bae30c0bbeb4b6acb408ae54f6c379
OUP	mango-bridge-contracts/solidity/Access/OwnableUpgradeable.sol	6111a9f9a910926a80916684ae7b5984201490d6
BCO	mango-bridge-contracts/solidity/Br	94af832764bf07e407ac297c81a82

	idgeConfig.sol	4dd6e00a716
DIERC1U	mango-bridge-contracts/solidity/interfaces/draft-IERC1822Upgradeable.sol	ebb0db653db26cc17742bc8047df2415b15209c6
IBL	mango-bridge-contracts/solidity/interfaces/IBridgeLimit.sol	ba96b9d658eb3bad319c7ff9b5ecd fac4bf9a8b3
IBC	mango-bridge-contracts/solidity/interfaces/IBridgeConfig.sol	031f1f08aa4f42412eedec1355ada d8df80e902f
IERC2	mango-bridge-contracts/solidity/interfaces/IERC20.sol	7452b23d71a2325d151a24aca940 0569a78024c3
ERC1P	mango-bridge-contracts/solidity/ERC1967Proxy.sol	91799ba1804c6dd4d37f354504eb 92715f6f5f8b
ECDSA	mango-bridge-contracts/solidity/ECDSA.sol	85e3cca22851e5592fbf295e4912df dc8db383c0
MBR	mango-bridge-contracts/solidity/MgoBridge.sol	168f8a242b1e6c1074eb00873133a b3837576501
BME	mango-bridge-contracts/solidity/BridgeMessage.sol	378adae6e3a3d617fe89a1e98253 4795f0ad16c9

1.3 Issue Statistic

Item	Count	Fixed	Acknowledged
Total	7	7	0
Informational	2	2	0
Minor	1	1	0
Medium	3	3	0
Major	0	0	0
Critical	1	1	0

1.4 MoveBit Audit Breakdown

ScaleBit aims to assess repositories for security-related issues, code quality, and compliance with specifications and best practices. Possible issues our team looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Integer overflow/underflow
- Number of rounding errors
- Unchecked External Call
- Unchecked CALL Return Values
- Functionality Checks
- Reentrancy
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic issues
- Gas usage
- Fallback function usage
- tx.origin authentication
- Replay attacks
- Coding style issues

1.5 Methodology

The security team adopted the "**Testing and Automated Analysis**", "**Code Review**" and "**Formal Verification**" strategy to perform a complete security test on the code in a way that is closest to the real attack. The main entrance and scope of security testing are stated in the conventions in the "Audit Objective", which can expand to contexts beyond the scope according to the actual testing needs. The main types of this security audit include:

(1) Testing and Automated Analysis

Items to check: state consistency / failure rollback / unit testing / value overflows / parameter verification / unhandled errors / boundary checking / coding specifications.

(2) Code Review

The code scope is illustrated in section 1.2.

(3) Audit Process

- Carry out relevant security tests on the testnet or the mainnet;
- If there are any questions during the audit process, communicate with the code owner in time. The code owners should actively cooperate (this might include providing the latest stable source code, relevant deployment scripts or methods, transaction signature scripts, exchange docking schemes, etc.);
- The necessary information during the audit process will be well documented for both the audit team and the code owner in a timely manner.

2 Summary

This report has been commissioned by [Mango](#) to identify any potential issues and vulnerabilities in the source code of the [Mango Bridge](#) smart contract, as well as any contract dependencies that were not part of an officially recognized library. In this audit, we have utilized various techniques, including manual code review and static analysis, to identify potential vulnerabilities and security issues.

During the audit, we identified 7 issues of varying severity, listed below.

ID	Title	Severity	Status
BRI-1	No Version Check	Minor	Fixed
CON-1	Not Strictly Checked	Medium	Fixed
SIG-1	Integer Truncation Leading to Reduced Signature Verification Security	Medium	Fixed
BRI1-1	Cross-chain Signature Replay	Medium	Fixed
SIG1-1	Signature Replay	Critical	Fixed
SIG1-2	Mismatch between <code>signer</code> and <code>pubkey</code>	Informational	Fixed
SIG1-3	Inconsistent Signature Verification Mechanism on Both Sides of the Bridge	Informational	Fixed

3 Participant Process

Here are the relevant actors with their respective abilities within the [Mango Bridge](#) Smart Contract :

Admin

- Admin can add the `TreasuryCap` through `add_treasury_cap<T>()` .
- Admin can claim the `TreasuryCap` through `claim_treasury_cap<T>()` .
- Admin can claim the bridge fee through `claim_bridge_fee<T>()` .
- Admin can set the bridge fee config through `op_bridge_fee<T>()` and `opFeeConfig()` .
- Admin can set the bridge limit through `op_bridge_limit<T>()` and `opTokenBridgeLimit()` .
- Admin can set the white and black list through `op_bridge_white_black()` and `opWhiteBlackList()` .
- Admin can set the signer and pubkey through `op_bridge_signers()` .
- Admin can set the fee receiver through `setFeeReceiver()` .
- Admin can set the bridge coinfig address through `setBridgeConfig()` .

User

- User can bridge their tokens to another chain through `bridge_token<T>()` and `bridgeToken()` .
- User can claim their tokens from another chain through `claim_token<T>()` and `claimToken()` .
- User can create a new tick and return the `TickRecord` object through `new_tick()` .
- User can create a new tick and share the `TickRecord` object through `coinscription_new_tick()` .
- User can mint a `CoinScription` with the `TickRecord` object through `do_mint()` and `coinscription_mint()` .
- User can merge their `CoinScription` through `do_merge()` and `coinscription_merge()` .
- User can split the `CoinScription` through `do_split()` , `split()` and `coinscription_split()` .

- User can burn the `CoinScription` with the `TickRecord` object through `do_burn()` and `coinscription_burn()` .
- User can transfer the `CoinScription` through `coinscription_transfer()` .
- User can create a new copyright and through `new_copyright()` and `singlescription_new_copyright()` .
- User can mint a `SingleScription` through `do_mint()` and `singlescription_mint()` .
- User can burn a `SingleScription` with the `SingleScriptionRecord` object through `do_burn` and `singlescription_burn()` .
- User can transfer the `SingleScription` through `singlescription_transfer()` .

4 Findings

BRI-1 No Version Check

Severity: Minor

Status: Fixed

Code Location:

mango-bridge-contracts/mango/sources/bridge.move#229-250;

mango-bridge-contracts/sui/sources/bridge.move#229-250

Descriptions:

`upload_claim_info` function has no version check

Suggestion:

It is recommended that adding version check

Resolution:

This issue has been fixed. The client has adopted our suggestions.

CON-1 Not Strictly Checked

Severity: Medium

Status: Fixed

Code Location:

mango-bridge-contracts/mango/sources/config.move#120-137;

mango-bridge-contracts/solidity/BridgeConfig.sol#134-142;

mango-bridge-contracts/sui/sources/config.move#120-137

Descriptions:

If the administrator initially adds only the minimum quantity limit and leaves the maximum quantity as 0 by default, then `max==0` will appear in the `can_bridge` function and the check will pass directly. The subsequent checks will not be executed and the minimum value check will be invalid.

Suggestion:

It is commanded that when there is no maximum value limit, it ensures that the minimum value check is passed.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

SIG-1 Integer Truncation Leading to Reduced Signature Verification Security

Severity: Medium

Status: Fixed

Code Location:

mango-bridge-contracts/mango/sources/sign.move#62-87;

mango-bridge-contracts/sui/sources/sign.move#62-87;

mango-bridge-contracts/solidity/BridgeConfig.sol#59-92

Descriptions:

In the `verify_signatures` function, the calculation of `min_signer_count` uses integer division ($\text{length} * 2 / 3$), which truncates the result (rounds down). This behavior can result in the required number of signers being calculated as less than the intended threshold. For example, when the length of `self.pubkeys` is 5, $5 * 2 / 3$ evaluates to 3, while the actual security requirement might be 4 signers. This precision loss lowers the security threshold for signature verification.

Suggestion:

It is recommended to replace the integer division logic with a ceiling calculation to ensure the required number of signers always meets the security requirements:

Resolution:

This issue has been fixed. The client has adopted our suggestions.

BRI1-1 Cross-chain Signature Replay

Severity: Medium

Status: Fixed

Code Location:

mango-bridge-contracts/sui/sources/bridge.move#313;

mango-bridge-contracts/mango/sources/bridge.move#313;

mango-bridge-contracts/solidity/MgoBridge.sol#112

Descriptions:

User calls the `claim_token` function to retrieve funds after cross-chain transactions. The cross-chain bridge supports 3 chains: Sui, Mango, and Ethereum.

The signature format on all 3 chains is identical, and the signature header constant is also the same: "MGO_BRIDGE_MESSAGE."

If an attacker replays the signature from the Sui chain to the Mango chain, it could result in a loss of funds from the cross-chain bridge.

However, since the project team employs some filtering mechanisms that are not visible in the contract, the severity of this vulnerability is moderately reduced.

Suggestion:

Modify the header constant "MGO_BRIDGE_MESSAGE" in the signature format so that each chain uses a different constant.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

SIG1-1 Signature Replay

Severity: Critical

Status: Fixed

Code Location:

mango-bridge-contracts/sui/sources/sign.move#90;

mango-bridge-contracts/mango/sources/sign.move#90;

mango-bridge-contracts/solidity/BridgeMessage.sol#24

Descriptions:

The user calls the `claim_token` function to retrieve funds after a cross-chain transaction.

The signature assembly method is shown in the following code.

Each field, when concatenated, does not include its length, which can lead to a signature replay vulnerability.

The signature message concatenation algorithm is as follows:

```
"MGO_BRIDGE_MESSAGE" + from_token(String) + bridge_hash(String) + typ(String) +  
destination(address) + amount_out(u64)
```

Assuming the original signature is as follows:

```
"MGO_BRIDGE_MESSAGE" + "usdt" + "0088aa" + "0x01::sui::Coin" + 0x3232 + 1
```

An attacker could rearrange the signature information as follows:

```
"MGO_BRIDGE_MESSAGE" + "usd" + "t0088aa" + "0x01::sui::Coin" + 0x3232 + 1
```

```
function encodeClaimMessage(  
    ClaimTokenMessage memory message  
) internal pure returns (bytes memory) {  
    return  
        abi.encodePacked(  
            MESSAGE_PREFIX,  
            message.fromToken,  
            message.bridgeHash,  
            message.toToken,  
            message.amountOut,  
            message.destination
```



```

        message.destination,
        message.amountOut
    );
}

```

```

fun encode_message_bytes(
    typ: String,
    destination: address,
    amount_out: u64,
    from_token: String,
    bridge_hash: String): vector<u8> {
    let from_token_bytes = into_bytes(from_token);
    let bridge_hash_bytes = into_bytes(bridge_hash);
    let to_token_bytes = into_bytes(typ);
    let destination_bytes = bcs::to_bytes(&destination);
    let amount_out_bytes = bcs::to_bytes(&amount_out);

    let message_bytes = MGO_MESSAGE_PREFIX;
    vector::append(&mut message_bytes, from_token_bytes);
    vector::append(&mut message_bytes, bridge_hash_bytes);
    vector::append(&mut message_bytes, to_token_bytes);
    vector::append(&mut message_bytes, destination_bytes);
    vector::append(&mut message_bytes, amount_out_bytes);
    message_bytes
}

```

Suggestion:

When concatenating the signature information, include the length of each field as part of the process.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

SIG1-2 Mismatch between `signer` and `pubkey`

Severity: Informational

Status: Fixed

Code Location:

mango-bridge-contracts/sui/sources/sign.move#47;

mango-bridge-contracts/mango/sources/sign.move#47

Descriptions:

The `op_signers` is used to set the signature verification for "signer" and "pubkey." However, there is no check to verify whether the "pubkey" corresponds to the provided "signer."

```
public(friend) fun op_signers(self: &mut BridgeSign, signer: address, pubkey: vector<u8>,
add: bool) {
    if (add) {
        assert!(!vector::contains(&self.addrs, &signer), ErrorSignerAlreadyExist);
        assert!(!vector::contains(&self.pubkeys, &pubkey), ErrorPubkeyAlreadyExist);
        vector::push_back(&mut self.addrs, signer);
        vector::push_back(&mut self.pubkeys, pubkey);
    } else {
        assert!(vector::contains(&self.addrs, &signer), ErrorSignerNoExist);
        assert!(vector::contains(&self.pubkeys, &pubkey), ErrorPubkeyNoExist);
        let (_, index) = vector::index_of(&self.addrs, &signer);
        vector::remove(&mut self.addrs, index);
        vector::remove(&mut self.pubkeys, index);
    }
}
```

Suggestion:

It is recommended to check the relationship between `signer` and `pubkey`: "signer = Hash(pubkey)".

Resolution:

This issue has been fixed. The client has adopted our suggestions.

SIG1-3 Inconsistent Signature Verification Mechanism on Both Sides of the Bridge

Severity: Informational

Status: Fixed

Code Location:

mango-bridge-contracts/sui/sources/sign.move#62;

mango-bridge-contracts/mango/sources/sign.move#62;

mango-bridge-contracts/solidity/BridgeConfig.sol#59

Descriptions:

The `verify_signatures<T>` function in the Move contract and the `validateMessage` function in the Solidity contract are both used for signature verification.

We have found that there is a slight difference in the signature verification between these two functions.

For example, if the "signer" set is (pub_A, pub_B, pub_C), the corresponding signatures are (sig_A, sig_B, sig_C).

In the `verify_signatures<T>` function, only any two or three elements from the (sig_A, sig_B, sig_C) set can appear, and there should be no repetition.

In the `validateMessage` function, combinations like (sig_A, sig_B, sig_B, sig_B, sig_A) are allowed, where the elements may repeat.

Suggestion:

Align the signature verification mechanisms on both sides of the cross-chain bridge to be consistent.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

Appendix 1

Issue Level

- **Informational** issues are often recommendations to improve the style of the code or to optimize code that does not affect the overall functionality.
- **Minor** issues are general suggestions relevant to best practices and readability. They don't post any direct risk. Developers are encouraged to fix them.
- **Medium** issues are non-exploitable problems and not security vulnerabilities. They should be fixed unless there is a specific reason not to.
- **Major** issues are security vulnerabilities. They put a portion of users' sensitive information at risk, and often are not directly exploitable. All major issues should be fixed.
- **Critical** issues are directly exploitable security vulnerabilities. They put users' sensitive information at risk. All critical issues should be fixed.

Issue Status

- **Fixed:** The issue has been resolved.
- **Partially Fixed:** The issue has been partially resolved.
- **Acknowledged:** The issue has been acknowledged by the code owner, and the code owner confirms it's as designed, and decides to keep it.

Appendix 2

Disclaimer

This report is based on the scope of materials and documents provided, with a limited review at the time provided. Results may not be complete and do not include all vulnerabilities. The review and this report are provided on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your own risk. A report does not imply an endorsement of any particular project or team, nor does it guarantee its security. These reports should not be relied upon in any way by any third party, including for the purpose of making any decision to buy or sell products, services, or any other assets. TO THE FULLEST EXTENT PERMITTED BY LAW, WE DISCLAIM ALL WARRANTIES, EXPRESS OR IMPLIED, IN CONNECTION WITH THIS REPORT, ITS CONTENT, RELATED SERVICES AND PRODUCTS, AND YOUR USE, INCLUDING BUT NOT LIMITED TO THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NOT INFRINGEMENT.

